

運用生成式 AI 和 Cloud Run 建構測驗產生器

來源：<https://codelabs.developers.google.com/cloud-genai?hl=zh-tw>

步驟數：12

在本程式碼研究室中，您將使用 Vertex AI，根據部分提供的規格設計益智問答測驗。您將在雲端託管的開發人員環境中測試測驗產生器，然後部署至 Google Cloud Run，即可公開發布該測驗。研究室結束時，您將整合測驗產生器與完整的應用程式。

目錄

1. 簡介
2. 設定
3. 提示 - 以自然語言程式設計
4. 建構簡易網路伺服器
5. 新增含有參數剖析功能的產生方法
6. 新增提示並設定格式
7. 新增 Vertex AI 用戶端程式庫
8. 雲端！
9. 整合所有資訊
10. 清除
11. 你辦到了！
12. 行動號召

步驟 1 · 約 3 分鐘

1. 簡介

在本實驗室中，您將建構網頁服務來產生益智測驗，並將其整合至有趣的實用應用程式。您會使用與以往不同的程式設計語言：英文！

執行步驟

- 您將根據一組條件製作提示，生成知識問答測驗。
- 您將建構簡單的網頁應用程式，並確認該應用程式在開發環境中如預期執行。
- 您會逐步在網路應用程式中新增邏輯，將其變成 API 伺服器，根據一組輸入參數產生測驗。
- 您將瞭解如何輕鬆使用 Google Cloud Run，將測驗生成服務部署至雲端。
- 最後，您將設定實際應用程式 (quizaic.com) 使用已部署的測驗產生器服務，並根據輸出內容進行即時測驗。

課程內容...

- 如何為大型語言模型 (LLM) 建立範本提示。
- 瞭解如何使用 Python 建立簡易的網路伺服器應用程式。
- 瞭解如何在網頁應用程式中加入 Google LLM 的支援功能。
- 瞭解如何將應用程式部署至雲端，讓任何人都能試用您的新作品。
- 如何將測驗產生器整合到較大的應用程式中。

事前準備

- Chrome 網路瀏覽器
- Google 帳戶
- 已啟用計費功能的 Cloud 專案

本實驗室適合各種程度的開發人員，包括初學者。雖然您會使用 Python，但不需要熟悉 Python 程式設計，就能瞭解發生了什麼事，因為我們會說明您看到的所有程式碼。

步驟 2 · 約 6 分鐘

2. 設定



本節說明如何開始進行這個實驗室。

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The image shows two screenshots of the Google Cloud console. The top screenshot shows the 'Select a project' dropdown menu in the top right corner of the Google Cloud interface. A blue arrow points from the dropdown to the 'NEW PROJECT' button. The bottom screenshot shows the 'New Project' form. The 'Project name' field is filled with 'My Project 13475'. The 'Project ID' field is filled with 'inbound-analogy-389614', which is highlighted with a blue box. The 'Location' field is empty, and there is a 'BROWSE' button next to it. At the bottom, there are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜

歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。

- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

- 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

在本實驗室中，您將在 [Cloud Shell](#) 工作階段中操作，這是由 Google Cloud 中執行的虛擬機器代管的指令解譯器。您也可以在本機電腦上輕鬆執行本節內容，但使用 Cloud Shell 可讓所有人在一致的環境中，獲得可重現的體驗。完成實驗室後，歡迎在自己的電腦上重試這個部分。

首次在 Cloud Shell 中提出需要授權的要求時，您可能會看到如下所示的「Authorize Cloud Shell」(授權 Cloud Shell) 對話方塊，請放心點選「Authorize」(授權) 按鈕。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

Authorize

Reject

啟用 Cloud Shell

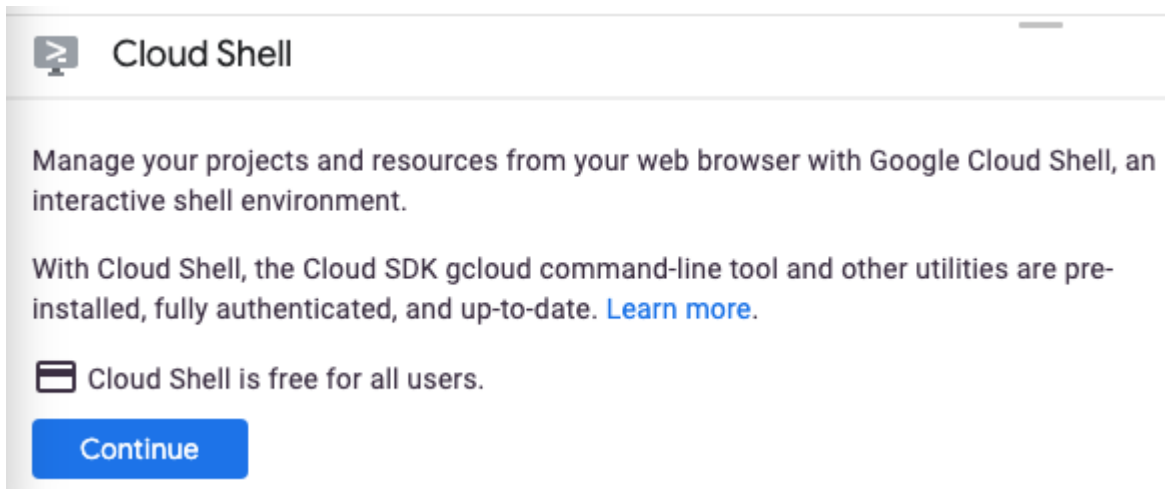
- 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



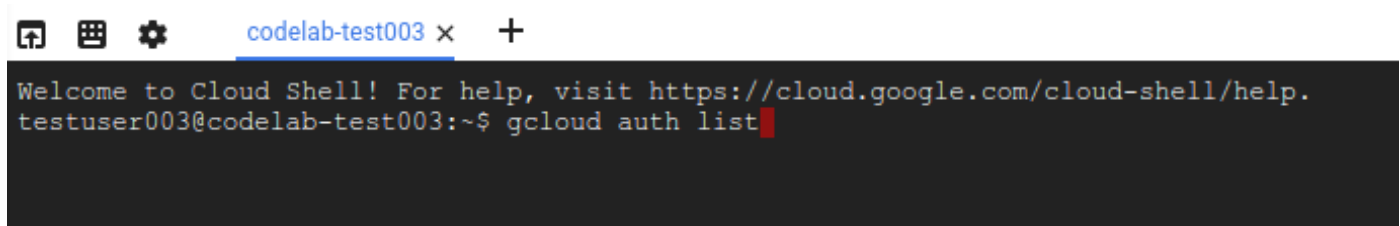
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

```
Credentialed Accounts
ACTIVE  ACCOUNT
*       <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。第一次執行指令時，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]  
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意：如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

啟用部分 API

在後續步驟中，您會瞭解需要這些服務的原因和用途，但現在請先執行下列指令，授予專案 Cloud Build、Artifact Registry、Vertex AI 和 Cloud Run 的存取權：

```
gcloud services enable cloudbuild.googleapis.com \  
                        artifactregistry.googleapis.com \  
                        aiplatform.googleapis.com \  
                        run.googleapis.com
```

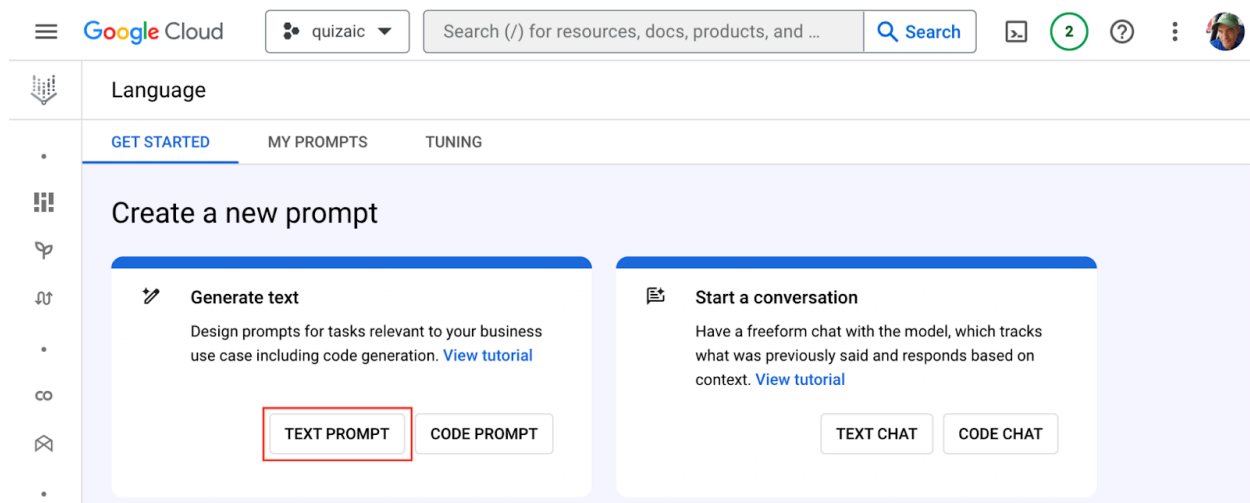
這時應該會顯示類似以下的成功訊息：

```
Operation "operations/acf.cc11852d-40af-47ad-9d59-477a12847c9e" finished  
successfully.
```

3. 提示 - 以自然語言程式設計



首先，我們要學習如何為大型語言模型開發提示。前往 Google Cloud 控制台 > Vertex AI > Vertex AI Studio (Language)。您應該會看到類似下列頁面的內容：



按一下「Generate Text」下方的「Text Prompt」按鈕。在下一個對話方塊中，輸入您認為可能有效生成知識問答測驗的提示，並符合下列需求：

- 主題：世界史
- 問題數量：5
- 難度：中等
- 語言：英文

按一下「提交」按鈕，即可查看輸出內容。

如下方螢幕截圖所示，您可以在右側面板選取要使用的模型，並微調部分設定：

←

Untitled prompt

FREEFORM

STRUCTURED

SAVE

<> VIEW CODE

▼ Prompt

Generate an intermediate level trivia quiz about World History.

▼ Response

Markdown

1. What was the name of the first European to reach the Americas?

A) Christopher Columbus
B) Leif Erikson
C) Vasco da Gama
D) Ferdinand Magellan

Answer: B) Leif Erikson

1 We want your [feedback](#).

Region
us-central1 (Iowa)

Model
text-bison (latest)

Temperature
0 1 0.2

> Advanced

SUBMIT

RESET PARAMETERS

REPORT INAPPROPRIATE RESPONSES

Content processed through Vertex AI is assessed against a list of safety attributes. Confidence scores for these attributes are returned in API responses [but are not visible in the UI](#).

可用的設定如下：

- 區域是生成要求應執行的位置。
- 「模型」：選取要使用的大型語言模型。在本程式碼研究室中，請使用「gemini-1.0-pro-001」。
- Temperature 會決定選取詞元的隨機程度。如果希望提示詞生成正確或適當的回覆，建議使用較低的溫度參數。如果溫度參數較高，則有可能生成較多樣化或預料之外的結果。
- 符記限制會決定針對單則提示輸出的文字數量上限。一個符記約為四個字元，預設值為 1024。
- 「前 K 個」會調整模型選取輸出符記的方式。如果「前 K 個」設為「1」，代表所選詞元是模型詞彙表所有詞元中可能性最高者 (也稱為「貪婪解碼」)。如果「前 K 個」設為「3」，則代表模型會依據 temperature，從可能性最高的 3 個詞元中選取下一個詞元。「前 K 個」的預設值為 40。
- Top-P 會影響模型選取輸出詞元的方式。模型會按照可能性最高到最低的順序選取符記，直到所選符記的可能性總和等於 Top-P 值。
- 「最多回覆」是指每則提示產生的模型回覆數量上限。
- 停止序列是一系列的字元 (包括空格)，如果模型遇到這個序列，就會停止產生回應。
- 「顯示正在產生的回應」選項可選擇是否要在生成回覆時顯示，或是儲存回覆並在完成後顯示。
- 安全篩選器門檻會調整看見可能有害回應的機率。

如果提示似乎能根據上述規定生成合理的測驗，我們可以使用自訂程式碼剖析這項測驗，但如果 LLM 能以結構化格式生成測驗，讓我們直接載入程式，豈不是更好？我們稍後會在實驗室中使用程式呼叫產生器，而該程式會以 JSON 格式表示測驗。JSON 是一種熱門的跨語言格式，用於表示結構化資料。

本實驗室中的測驗會以物件陣列表示，每個物件都包含問題、該問題的可能回覆陣列，以及正確回覆。以下是本實驗室中測驗的 JSON 編碼：

```
[
  {
    "question": "Who was the first person to walk on the moon?",
    "responses": [
      "Neil Armstrong",
      "Buzz Aldrin",
      "Michael Collins",
      "Yuri Gagarin"
    ],
    "correct": "Neil Armstrong"
  },
  {
    "question": "What was the name of the war that took place between the British and the French in North America from 1754 to 1763??",
    "responses": [
      "The French and Indian War",
      "The Seven Years' War",
      "The War of the Austrian Succession",
      "The Great War"
    ],
    "correct": "The French and Indian War"
  },
  ...
]
```

看看是否可以修改提示，讓測驗以所需的 JSON 格式輸出。

提示：如果這個步驟有困難，以下提供兩種可能會有幫助的選項 (兩者都適用)：

1. 以文字指定所需格式 (例如上方的斜體句子)。
2. 在提示中加入所需 JSON 格式的範例。

提示可根據所需規格生成測驗後，請按一下頁面右上角的 `GET CODE` 按鈕，查看可用於以程式輔助方式將提示提交至 Vertex AI LLM 的 Python 程式碼。如果您想使用 Python 以外的程式設計語言，請參閱 <https://cloud.google.com/vertex-ai/docs/samples?text=generative>。

如果無法想出令人滿意的提示，請別緊張。您會在後續步驟中花更多時間處理這項作業，如果需要協助，我們也會在稍後提供提示。

步驟 4 · 約 8 分鐘

4. 建構簡易網路伺服器



現在您已建立可用的提示，接下來要將提示整合到較大的應用程式中。當然，我們可以將提示嵌入較大應用程式的原始碼，但我們希望產生器能做為微服務，為其他應用程式提供測驗產生服務。為此，我們需要建立簡易網路伺服器，並對外公開。我們會在後續步驟中執行這項操作。

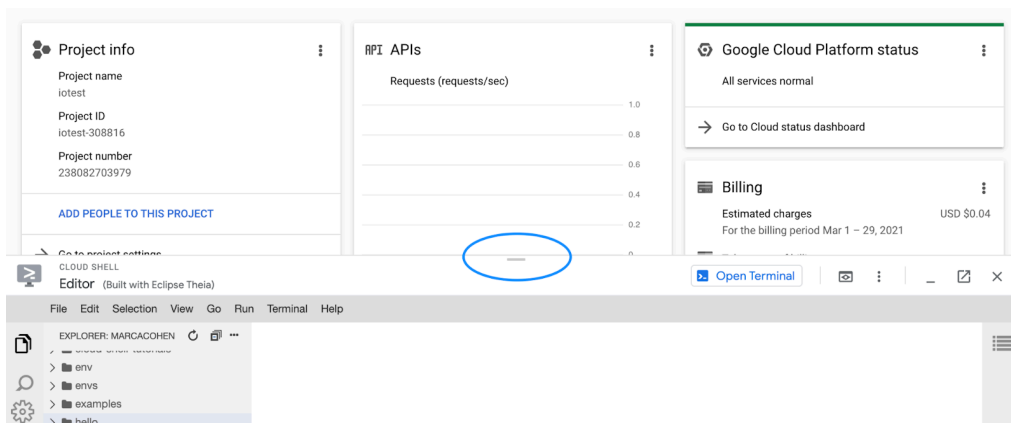
首先，請點選 Cloud Shell 面板頂端的 **Open Editor** 按鈕。這是訂閱按鈕的圖示：



接著，您會進入類似 Visual Studio Code 的 IDE 環境，可在其中建立專案、編輯原始碼及執行程式等。

在本實驗室中，請使用新版編輯器。如果看到「試用新版編輯器」按鈕，表示你目前使用的是舊版編輯器，請按一下該按鈕切換至新版編輯器。

如果畫面太過擁擠，可以拖曳控制台與編輯/終端機視窗之間的分隔線 (如下圖醒目顯示的水平線)，展開或縮小這兩個區域：



如要在編輯器和終端機之間切換，請分別點選 **Open Editor** 和 **Open Terminal** 按鈕。現在請嘗試在這兩種環境之間來回切換。

此外，還有「在新視窗中開啟」按鈕 (如下所示：



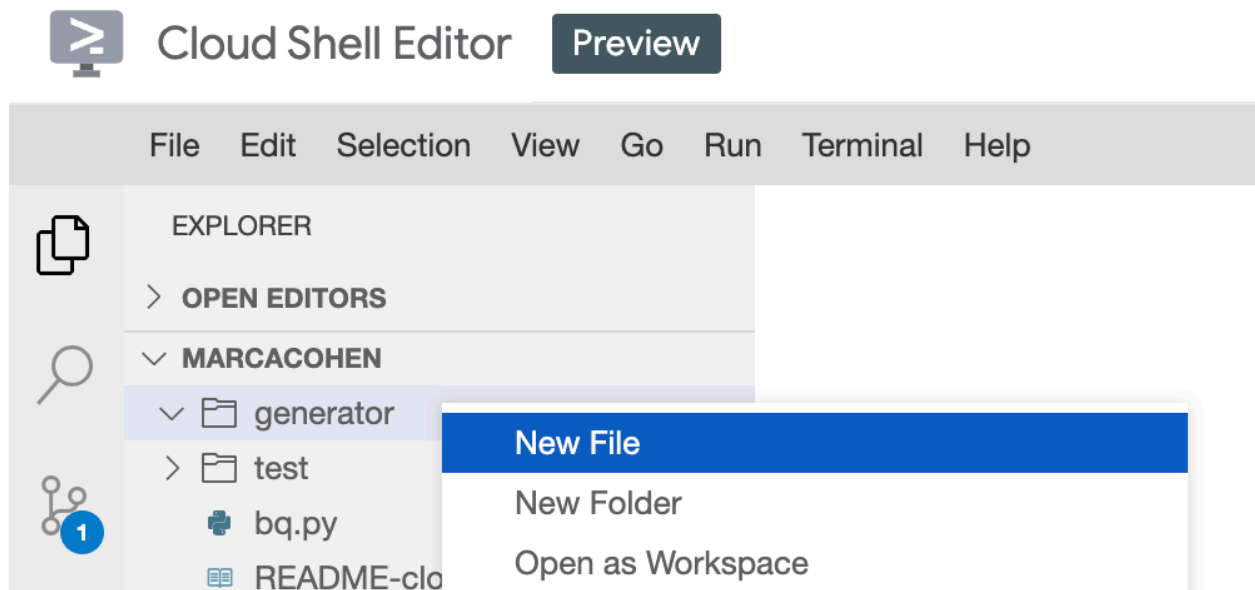
)，可將 Cloud 控制台和終端機/編輯器分別顯示在不同視窗中。本實驗室不需要使用該功能，但瞭解這項功能很有幫助。

接著，按一下「新增資料夾」按鈕



，輸入 `quiz-generator`，然後按下 Enter 鍵，建立資料夾來儲存這個實驗室的作業。您在本實驗室中建立的所有檔案，以及在 Cloud Shell 中執行的所有工作，都會在這個資料夾中進行。

現在請建立 `requirements.txt` 檔案。這會告知 Python 應用程式依附的程式庫。在這個簡單的網頁應用程式中，您將使用熱門的 Python 模組 (稱為 `Flask`，用戶端程式庫) 建構網路伺服器，並使用稱為 `gunicorn` 的網路伺服器架構。`google-cloud-aiplatform` 在檔案導覽窗格中，在 `quiz-generator` 資料夾上按一下滑鼠右鍵，然後選取 `New file` 選單項目，如下所示：



系統提示輸入新檔案名稱時，請輸入 `requirements.txt`，然後按下 Enter 鍵。確認新檔案最終會出現在 `quiz-generator` 專案資料夾中。

將下列程式碼行貼到新檔案中，指定應用程式依附於 Python `flask` 套件、`gunicorn` 網頁伺服器和 `google-cloud-aiplatform` 用戶端程式庫，以及每個項目的相關聯版本。

```
flask==3.0.0
gunicorn==21.2.0
google-cloud-aiplatform==1.47.0
```

您不必明確儲存這個檔案，因為 Cloud Editor 會自動儲存變更。

使用相同技巧，建立另一個名為 `main.py` 的新檔案。這會是應用程式的主要 (也是唯一) Python 來源檔案。同樣地，請確認新檔案最終會出現在 `quiz-generator` 資料夾中。

將下列程式碼插入這個檔案：

```
from flask import Flask
import os

app = Flask(__name__) # Create a Flask object.
PORT = os.environ.get("PORT") # Get PORT setting from environment.
if not PORT:
    PORT = 8080

# The app.route decorator routes any GET requests sent to the root path
# to this function, which responds with a "Hello world!" HTML document.
@app.route("/", methods=["GET"])
def say_hello():
    html = "<h1>Hello world!</h1>"
    return html

# This code ensures that your Flask app is started and listens for
# incoming connections on the local interface and port 8080.
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=PORT)
```

請詳閱行內註解（「#」字元後的文字），瞭解每行 Python 程式碼的作用。

切換回終端機，並使用下列指令切換至專案資料夾：

```
cd quiz-generator
```

如果發現 Cloud Shell 無法正常運作，可能是因為系統重新啟動，並返回主目錄。輸入 `pwd`（列印工作目錄）即可查看目前是否位於主目錄。如果是，請執行 `cd quiz-generator` 指令，切換至專案資料夾。

執行下列指令來安裝專案依附元件：

```
pip3 install -r requirements.txt
```

安裝依附元件後，畫面會顯示類似以下的輸出：

```
Successfully installed flask-3.0.0
```

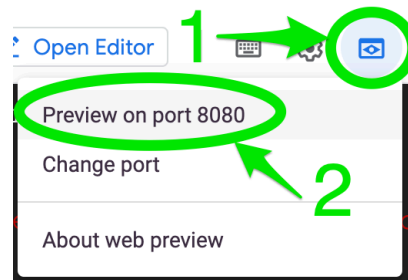
現在在終端機中執行下列指令，即可啟動應用程式：

```
flask --app main.py --debug run --port 8080
```

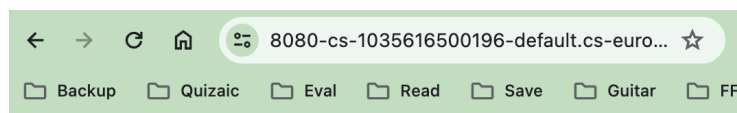
前述指令列中的 `--debug` 選項會讓 Flask 網頁伺服器在您變更任何來源檔案時重新載入。也就是說，我們變更 `main.py` 時，不需要重新啟動伺服器。

此時，您的應用程式會在專為 Cloud Shell 工作階段而設的虛擬機器上執行。Cloud Shell 內含 Proxy 機制，可讓您從全球網際網路的任何位置，存取虛擬機器上執行的網路伺服器 (例如您剛啟動的伺服器)。

依序點選 `web preview` 按鈕和 `Preview on Port 8080` 選單項目，如下所示：



系統會開啟網路瀏覽器分頁，顯示執行中的應用程式，畫面應如下所示：



Hello world!

5. 新增含有參數剖析功能的產生方法

現在我們想新增對 `generate` 這個新方法的支援。方法是在 HTTP 要求中加入匯入陳述式，並修改主要路徑來剖析這項要求並列印參數，如下所示：

系統會以 `#<-CHANGED` 註解醒目顯示新增和修改的程式碼行。您可以選擇性地進行這些變更，也可以將整個 `main.py` 內容替換為下列程式碼。

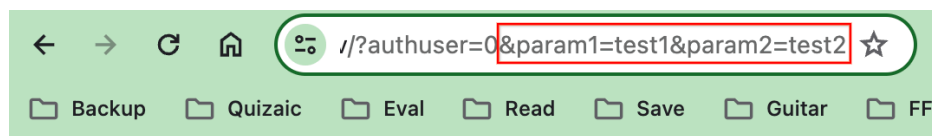
```
from flask import Flask
from flask import request                                #<-CHANGED
import os

app = Flask(__name__) # Create a Flask object.
PORT = os.environ.get("PORT") # Get PORT setting from environment.
if not PORT:
    PORT = 8080

# The app.route decorator routes any GET requests sent to the /generate
# path to this function, which responds with "Generating:" followed by
# the body of the request.
@app.route("/", methods=["GET"])                        #<-CHANGED
def generate():                                         #<-CHANGED
    params = request.args.to_dict()                    #<-CHANGED
    html = f"<h1>Quiz Generator</h1>"                 #<-CHANGED
    for param in params:                               #<-CHANGED
        html += f"<br>{param}={params[param]}"         #<-CHANGED
    return html                                         #<-CHANGED

# This code ensures that your Flask app is started and listens for
# incoming connections on the local interface and port 8080.
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=PORT)
```

現在重新載入現有的網路瀏覽器分頁，即可查看結果。這次您應該會看到「測驗產生器」，以及系統自動在網址中加入的查詢參數 (`authuser`)。請在瀏覽器網址列的網址尾端附加字串 `¶m1=val1¶m2=val2`，然後重新載入頁面，您應該會看到類似下方的內容：



Quiz Generator

```
authuser=0
param1=test1
param2=test2
```

我們已瞭解如何在網址上傳送及剖析查詢參數，現在要新增對特定參數的支援，以便傳送至測驗產生器，這些參數如下：

- `topic` - 測驗主題
- `num_q` - 想要的問題數量
- `diff` - 想要的難度等級 (簡單、中等、困難)
- `lang` : 所需測驗語言

系統會以 `#<-CHANGED` 註解醒目顯示新增和修改的程式碼行。您可以選擇性地進行這些變更，也可以將整個 `main.py` 內容替換為下列程式碼。


```

from flask import Flask
from flask import request
import os

# Default quiz settings #<-CHANGED
TOPIC = "History"      #<-CHANGED
NUM_Q = "5"           #<-CHANGED
DIFF = "intermediate" #<-CHANGED
LANG = "English"      #<-CHANGED

app = Flask(__name__) # Create a Flask object.
PORT = os.environ.get("PORT") # Get PORT setting from environment.
if not PORT:
    PORT = 8080

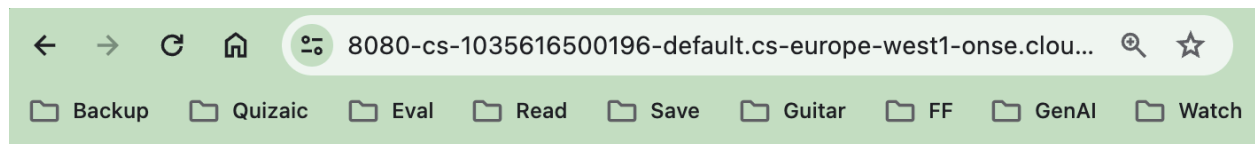
# This function takes a dictionary, a name, and a default value.
# If the name exists as a key in the dictionary, the corresponding
# value is returned. Otherwise, the default value is returned.
def check(args, name, default): #<-CHANGED
    if name in args:            #<-CHANGED
        return args[name]      #<-CHANGED
    return default              #<-CHANGED

# The app.route decorator routes any GET requests sent to the /generate
# path to this function, which responds with "Generating:" followed by
# the body of the request.
@app.route("/", methods=["GET"])
# This function generates a quiz using Vertex AI.
def generate():
    args = request.args.to_dict() #<-CHANGED
    topic = check(args, "topic", TOPIC) #<-CHANGED
    num_q = check(args, "num_q", NUM_Q) #<-CHANGED
    diff = check(args, "diff", DIFF) #<-CHANGED
    lang = check(args, "lang", LANG) #<-CHANGED
    html = f"""
        <h1>Quiz Generator</h1><br>
        {topic}<br>
        {num_q}<br>
        {diff}<br>
        {lang}""" #<-CHANGED
    return html

# This code ensures that your Flask app is started and listens for
# incoming connections on the local interface and port 8080.
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=PORT)

```

現在重新載入現有的網路瀏覽器分頁，即可查看結果。您應該會看到類似下列的網頁：

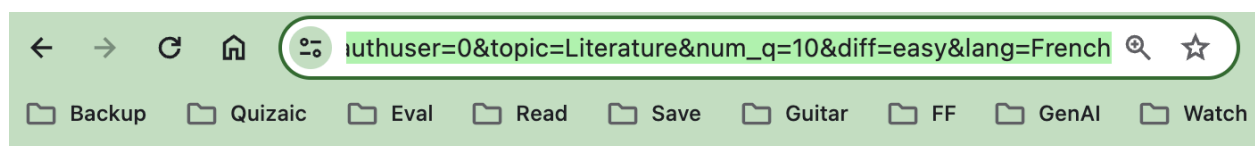


Quiz Generator

```
topic='History'  
num_q='5'  
diff='intermediate'  
lang='English'
```

嘗試變更網址，為各種參數設定值。舉例來說，請在網址列的網址結尾加上尾碼「?

authuser=0&topic=Literature&num_q=10&diff=easy&lang=French」：



Quiz Generator

```
topic='Literature'  
num_q='10'  
diff='easy'  
lang='French'
```

6. 新增提示並設定格式

接著，我們要為想傳送給測驗產生器的特定參數新增支援功能，這些參數如下：

- `topic` - 測驗主題
- `num_q` - 想要的問題數量
- `diff` - 想要的難度等級 (簡單、中等、困難)
- `lang` : 所需測驗語言

複製您在稍早步驟中透過 Vertex Generative AI Studio 開發的提示，但將主題、問題數量和難度等級的硬式編碼值換成下列字串：

- `{topic}`
- `{num_q}`
- `{diff}`
- `{lang}`

系統會以 `#<-CHANGED` 註解醒目顯示新增和修改的程式碼行。您可以選擇性地進行這些變更，也可以將整個 `main.py` 內容替換為下列程式碼。

```

from flask import Flask
from flask import request
import os

# Default quiz settings
TOPIC = "History"
NUM_Q = 5
DIFF = "intermediate"
LANG = "English"

PROMPT = ""
Generate a quiz according to the following specifications:

- topic: {topic}
- num_q: {num_q}
- diff: {diff}
- lang: {lang}

Output should be (only) an unquoted json array of objects with keys:
"Question", "responses", and "correct".

""" #<-CHANGED

app = Flask(__name__) # Create a Flask object.
PORT = os.environ.get("PORT") # Get PORT setting from environment.
if not PORT:
    PORT = 8080

# This function takes a dictionary, a name, and a default value.
# If the name exists as a key in the dictionary, the corresponding
# value is returned. Otherwise, the default value is returned.
def check(args, name, default):
    if name in args:
        return args[name]
    return default

# The app.route decorator routes any GET requests sent to the /generate
# path to this function, which responds with "Generating:" followed by
# the body of the request.
@app.route("/", methods=["GET"])
# This function generates a quiz using Vertex AI.
def generate():
    args = request.args.to_dict()
    topic = check(args, "topic", TOPIC)
    num_q = check(args, "num_q", NUM_Q)
    diff = check(args, "diff", DIFF)
    lang = check(args, "lang", LANG)
    prompt = PROMPT.format(topic=topic, num_q=num_q, diff=diff, lang=lang) #<-CHANGED
    html = f"<h1>Prompt:</h1><br><pre>{prompt}</pre>" #<-CHANGED
    return html

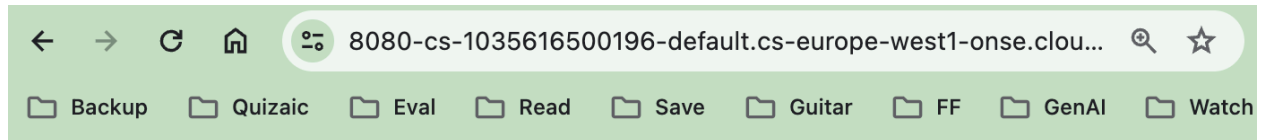
# This code ensures that your Flask app is started and listens for

```

```
# incoming connections on the local interface and port 8080.
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=PORT)
```

在這個簡單的例子中，提示會嵌入多行 Python 字串。在更實際的案例中，您可能會想將提示保留在外部文字檔中，並將內容讀取到 `main.py` 檔案的變數中。

現在重新載入現有的網路瀏覽器分頁，即可查看結果。您應該會看到類似下列的網頁：



Prompt:

Generate a quiz according to the following specifications:

- topic: Literature
- num_q: 10
- diff: easy
- lang: French

Output should be (only) an unquoted json array of objects with keys: "Question", "responses", and "correct".

請嘗試修改網址，變更這四個參數。

步驟 7 · 約 8 分鐘

7. 新增 Vertex AI 用戶端程式庫

現在可以開始使用 [Vertex AI Python 用戶端程式庫](#) 生成測驗。這會自動執行步驟 3 中的互動式提示，並讓生成器服務以程式輔助方式存取 Google 的 LLM 功能。請按照下列方式更新 `main.py` 檔案：

系統會以 `#<-CHANGED` 註解醒目顯示新增和修改的程式碼行。您可以選擇性地進行這些變更，也可以將整個 `main.py` 內容替換為下列程式碼。

請務必將「YOUR_PROJECT」替換為實際的專案 ID。

```

from flask import Flask
from flask import request
from flask import Response                                     #<-CHANGED
import os

import vertexai
from vertexai.generative_models import GenerativeModel      #<-CHANGED

# Default quiz settings
TOPIC = "History"
NUM_Q = 5
DIFF = "intermediate"
LANG = "English"
MODEL = "gemini-1.0-pro"  #<-CHANGED

PROMPT = ""
Generate a quiz according to the following specifications:

- topic: {topic}
- num_q: {num_q}
- diff:  {diff}
- lang:  {lang}

Output should be (only) an unquoted json array of objects with keys "question", "responses",
and "correct".

"""

app = Flask(__name__) # Create a Flask object.
PORT = os.environ.get("PORT") # Get PORT setting from environment.
if not PORT:
    PORT = 8080

# Initialize Vertex AI access.
vertexai.init(project="YOUR_PROJECT", location="us-central1") #<-CHANGED
parameters = {                                                #<-CHANGED
    "candidate_count": 1,                                     #<-CHANGED
    "max_output_tokens": 1024,                               #<-CHANGED
    "temperature": 0.5,                                       #<-CHANGED
    "top_p": 0.8,                                             #<-CHANGED
    "top_k": 40,                                              #<-CHANGED
}                                                            #<-CHANGED
model = GenerativeModel(MODEL)                                #<-CHANGED

# This function takes a dictionary, a name, and a default value.
# If the name exists as a key in the dictionary, the corresponding
# value is returned. Otherwise, the default value is returned.
def check(args, name, default):
    if name in args:
        return args[name]
    return default

```

```
# The app.route decorator routes any GET requests sent to the /generate
# path to this function, which responds with "Generating:" followed by
# the body of the request.
@app.route("/", methods=["GET"])
# This function generates a quiz using Vertex AI.
def generate():
    args = request.args.to_dict()
    topic = check(args, "topic", TOPIC)
    num_q = check(args, "num_q", NUM_Q)
    diff = check(args, "diff", DIFF)
    lang = check(args, "lang", LANG)
    prompt = PROMPT.format(topic=topic, num_q=num_q, diff=diff, lang=lang)
    response = model.generate_content(prompt, generation_config=parameters) #<-CHANGED
    print(f"Response from Model: {response.text}") #<-CHANGED
    html = f"{response.text}" #<-CHANGED
    return Response(html, mimetype="application/json") #<-CHANGED

# This code ensures that your Flask app is started and listens for
# incoming connections on the local interface and port 8080.
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=PORT)
```

現在重新載入現有的網路瀏覽器分頁，即可查看結果。請注意，由於您現在實際提出 LLM 要求，這可能需要幾秒鐘的時間。您應該會看到類似下列的網頁：



```
[
  {
    "question": "What was the name of the first European to reach the Americas?",
    "responses": [
      "Christopher Columbus",
      "Leif Erikson",
      "Amerigo Vespucci",
      "Ferdinand Magellan"
    ],
    "correct": "Leif Erikson"
  },
  {
    "question": "What was the name of the first permanent English settlement in the Americas?",
    "responses": [
      "Jamestown",
      "Plymouth",
      "Boston",
      "New York"
    ],
    "correct": "Jamestown"
  }
],
```

嘗試變更網址，要求其他測驗主題、問題數量和難度。

微服務就此完成，恭喜！在下一個步驟中，您將瞭解如何將服務部署至雲端，讓任何人都能隨時隨地存取。

步驟 8 · 約 8 分鐘

8. 雲端！



您已建構自己的測驗產生器，現在要將這項絕妙工具部署至雲端，與全世界分享。但你希望的不只是分享，請務必確認：

- 穩定執行：如果執行應用程式的電腦當機，您會自動獲得容錯能力
- 自動調度資源：應用程式可因應大量流量，並在閒置時自動減少資源用量
- 盡量減少費用，因為系統不會對您未使用的資源收費，只會對處理流量時耗用的資源收費
- 可透過自訂網域名稱存取 - 您可以一鍵將自訂網域名稱指派給服務
- 提供優異的回應時間 - 冷啟動的回應速度合理，但您可以指定[執行個體數量下限](#)設定，進一步微調回應速度
- 使用標準 SSL/TLS 網路安全防護機制支援端對端加密 - 部署服務時，您會自動免費取得標準網路加密功能和相應的必要憑證

將應用程式部署至 Google Cloud Run，即可享有上述所有優點，以及更多好處。如要透過 Cloud Run 共用應用程式，基本建構區塊就是容器。

容器可讓我們建立模組化方塊，在其中執行應用程式，並將所有依附元件組合在一起。由於容器幾乎可在任何虛擬或實體伺服器上使用，因此我們可讓您在任何位置部署應用程式，從內部部署到雲端，甚至將應用程式從一個服務供應商移至另一個服務供應商。

如要進一步瞭解容器，以及容器在 Google Cloud Run 中的運作方式，請參閱「[透過 Cloud Run 輕鬆三步驟，從開發環境到正式環境](#)」程式碼研究室。

將應用程式部署至 Cloud Run

Cloud Run 是區域服務，這表示執行 Cloud Run 服務的基礎架構位於特定區域，並由 Google 代管，可為該區域內所有可用區提供備援功能。為求簡單，在本實驗室中，我們將使用硬式編碼的 `us-central1` 區域。

如需目前支援的區域清單，請參閱「[Cloud Run \(全代管\) 地區](#)」。

我們將使用所謂的建構包，自動產生容器。在 Cloud Editor 中建立名為 `Procfile` 的新檔案，然後插入這行文字：

```
web: gunicorn --bind :$PORT --workers 1 --threads 8 --timeout 0 main:app
```

這會告知建構包系統如何在自動產生的容器中執行應用程式。接著，在 Cloud Shell 終端機 (位於同一個 `quiz-generator` 目錄) 中執行下列指令：

```
gcloud run deploy quiz-generator \
  --source . \
  --region us-central1 \
  --allow-unauthenticated
```

系統提示在 Artifact Registry 建立 Docker 存放區時，請回答「是」。

這會告知 `gcloud` 指令，您希望使用 Buildpacks 根據目前目錄中找到的來源檔案 (`--source .` 中的 `dot` 是目前目錄的簡寫) 建立容器映像檔。由於服務會隱含地處理容器映像檔，因此您不必在這個 `gcloud` 指令中指定映像檔。

如要進一步瞭解部署 Cloud Run 服務時可用的選項，請輸入 `gcloud run deploy --help`。

稍候片刻，等待部署作業完成。成功後，`gcloud` 指令會顯示新服務的網址：

```
Building using Buildpacks and deploying container to Cloud Run service [quiz-
generator] in project [YOUR_PROJECT] region [YOUR_REGION]
OK Building and deploying new service... Done.
  OK Creating Container Repository...
  OK Uploading sources...
  OK Building Container... Logs are available at
[https://console.cloud.google.com/cloud-build/builds/0cf1383f-35db-412d
-a973-557d5e2cd4a4?project=780573810218].
  OK Creating Revision...
  OK Routing traffic...
  OK Setting IAM Policy...
Done.
Service [quiz-generator] revision [quiz-generator-00001-xnr] has been deployed and is
serving 100 percent of traffic.
Service URL: https://quiz-generator-co24gukjmq-uc.a.run.app
```

您也可以使用下列指令擷取服務網址：

```
gcloud run services describe quiz-generator \
  --region us-central1 \
  --format "value(status.url)"
```

畫面應如下所示：

```
https://quiz-generator-co24gukjmq-uc.a.run.app
```

這個連結是 Cloud Run 服務專用的網址，並採用 TLS 安全性。這個連結是永久連結 (只要您未停用服務)，可在網路上任何位置使用。這項功能不會使用先前提及的 Cloud Shell Proxy 機制，該機制依附於暫時性虛擬機器。

按一下醒目顯示的 `Service URL`，開啟網路瀏覽器分頁，查看正在執行的應用程式。確認結果與開發環境中顯示的結果相同。此外，請確認您可以在網址結尾提供參數，調整產生的測驗。

恭喜！您的應用程式現已在 Google Cloud 中執行。您的應用程式會自動公開發布，並採用 TLS (HTTPS) 加密，還能自動擴充，因應驚人的流量。

步驟 9 · 約 8 分鐘

9. 整合所有資訊



最後一個步驟是執行測驗產生器，這是 quizaic 應用程式的一部分。請前往 [quizaic 網址](#)，登入 Google 帳戶，然後前往 **Create Quiz** 分頁。選取產生器類型 **Custom**，將 Cloud Run 網址貼到「網址」欄位，填寫其他必填欄位，然後提交表單。

 Quizaic | AI Powered Infinite Trivia 

 Browse

 Create

 Play

Create Quiz

Quiz Name

Quiz Generator

Custom Generator URL

Enter free-form text for Custom generator topic

Answer Format

Number of Questions

Difficulty

Quiz Language

Save Quiz

The image shows the Quizaic app interface. At the top, there's a header with the Quizaic logo and the text "AI Powered Infinite Trivia". Below the header, there's a sidebar with three main options: "Browse" (represented by a grid icon), "Create" (represented by a plus icon), and "Play" (represented by a game controller icon). The main area displays four trivia quizzes in a 2x2 grid. Each quiz card has a title, a featured image, and four action buttons: "Host", "Edit", "Clone", and "Delete". The quizzes are: 1. "Pop Music" with a vintage microphone image. 2. "Cat Trivia by حسين رضايي" with a black and white cat image. 3. "US History Quiz" with a large crowd at a historical site image. 4. "My new quiz" with a portrait of Benjamin Franklin image.

10. 清除



不使用服務時，Cloud Run 不會收費，但您可能仍須支付儲存所建構之容器映像檔的費用。

如要避免產生費用，您可以刪除 GCP 專案，這樣一來，系統就會停止對該專案中使用的所有資源計費。此外，也可以直接執行下列指令刪除容器映像檔：

```
gcloud config set artifacts/repository cloud-run-source-deploy
gcloud config set artifacts/location us-central1
gcloud artifacts docker images list

# Note image tag for resulting list

gcloud artifacts docker images delete <IMAGE-TAG>
```

如要刪除 Cloud Run 服務，請使用下列指令：

```
gcloud run services delete quiz-generator --region us-central1 --quiet
```

11. 你辦到了！



恭喜！您已成功製作 LLM 提示，並使用該提示部署 Cloud Run 微服務。現在你可以用自然語言編寫程式，並與全世界分享你的作品！

最後，我想請您思考一個重要問題：

在開發人員環境中讓應用程式正常運作後，您必須修改多少程式碼，才能將應用程式部署至雲端，並享有 **Cloud Run** 提供的所有生產環境等級屬性？

答案當然是零。:)

如需其他語言的 Cloud Run 入門指南，請參閱以下網頁：<https://cloud.google.com/run/docs/quickstarts/build-and-deploy>。您可以在這裡找到 Go、Node.js、Java、C#、PHP、Ruby、Shell 指令碼和其他語言的操作說明。

其他程式碼研究室...

- [透過 Cloud Run，輕鬆三步驟即可將開發環境部署至正式環境](#)
- [使用 Vertex AI 和 Svelte Kit 的文字摘要應用程式](#)
- [在 Cloud Run 上使用 PaLM API 建構對話應用程式](#)
- [包裝 PaLM Text Bison 模型的 Cloud Function](#)
- [使用 Spanner 和 Vertex AI Imagen API 將資料轉換為生成式 AI](#)

參考文件...

- [Vertex AI 說明文件](#)
- [瞭解大型語言模型、PaLM 模型和 Vertex AI](#)
- [Cloud Run 說明文件](#)
- [開發 Cloud Run 服務](#)
- [建構容器](#)
- [非官方 Cloud Run 常見問題](#)
- [Cloud Run 定價](#)

步驟 12 · 約 1 分鐘

12. 行動號召

如果您喜歡這個程式碼研究室，而且可能會花更多時間實際使用 Google Cloud，那麼您應該立即[加入 Google Cloud Innovators](#)！



Google Cloud Innovators 免費，並提供以下福利：

- 參加即時討論、AMA 和發展藍圖講座，直接取得來自 Google 員工的最新資訊。
- 直接在收件匣中掌握最新的 **Google Cloud** 消息
- 數位徽章和視訊會議背景
- 價值 \$500 美元的抵免額，用於 Skills Boost 的研究室和課程

按[這裡](#)報名！
